

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»
(Университет ИТМО)

Факультет **Прикладной информатики**
Образовательная программа **Мобильные и сетевые технологии**
Направление подготовки(специальность) **09.03.03 Прикладная информатика**

ЛАБОРАТОРНАЯ РАБОТА №6

По дисциплине «Проектирование и реализация баз данных»

Тема: Введение в СУБД MongoDB. Установка MongoDB. Начало работы с БД. Работа с БД в СУБД MongoDB.

Выполнил Барецкий М.С.; К3241.
Проверил Говорова М.М.

Санкт-Петербург 2026

1 Цель работы

- 1) Овладеть практическими навыками установки СУБД MongoDB.
- 2) Овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

2 Практические задания

Практическое задание для ЛР 6.1:

- Установите MongoDB для обеих типов систем (32/64 бита).
- Проверьте работоспособность системы запуском клиента mongo.
- Выполните методы:
 - a) `db.help()`
 - b) `db.help`
 - c) `db.stats()`
- Создайте БД learn.
- Получите список доступных БД.
- Создайте коллекцию unicorns, вставив в нее документ `{name: 'Aurora', gender: 'f', weight: 450}`.
- Просмотрите список текущих коллекций.
- Переименуйте коллекцию unicorns.
- Просмотрите статистику коллекции.
- Удалите коллекцию.
- Удалите БД learn.

Практическое задание для ЛР 6.2: см. пункт 5 “Выполнение лабораторной работы №6.2”

4 Выполнение лабораторной работы №6.1

4.1 Установка СУБД MongoDB

Для выполнения лабораторной работы использовался пакетный менеджер Homebrew. С его помощью была установлена утилита MongoDB Atlas CLI, предназначенная для управления облачными кластерами MongoDB Atlas.

Установка выполнялась командой:

```
brew install mongodb-community
```

4.2 Вызов базовых методов и создание БД

Для просмотра списка доступных методов объекта базы данных использовалась команда:

```
db.help()
```

Команда отображает основные функции и методы для работы с MongoDB.

```
[test> db.version()
8.2.9
[test> db.help()

Database Class:
getMongo           Returns the current database connection
getName           Returns the name of the DB
getCollectionNames Returns an array containing the names of all collections in the current database.
getCollectionInfos Returns an array of documents with collection information, i.e. collection name and options, for the current database.
runCommand         Runs an arbitrary command on the database.
adminCommand       Runs an arbitrary command against the admin database.
aggregate          Runs a specified admin/diagnostic pipeline which does not require an underlying collection.
getSiblingDB       Returns another database without modifying the db variable in the shell environment.
getCollection       Returns a collection or a view object that is functionally equivalent to using the db.<collectionName>.
dropDatabase       Removes the current database, deleting the associated data files.
createUser         Creates a new user for the database on which the method is run. db.createUser() returns a duplicate user error if the user
already exists on the database.
updateUser         Updates the user's profile on the database on which you run the method. An update to a field completely replaces the previous field's values. This includes updates to the user's roles array.
changeUserPassword Updates a user's password. Run the method in the database where the user is defined, i.e. the database you created the user
.
```

Рис. 1: Вызов метода db.help()

```
[test> db.stats()
{
  db: 'test',
  collections: Long('0'),
  views: Long('0'),
  objects: Long('0'),
  avgObjSize: 0,
  dataSize: 0,
  storageSize: 0,
  indexes: Long('0'),
  indexSize: 0,
  totalSize: 0,
  scaleFactor: Long('1'),
  fsUsedSize: 0,
  fsTotalSize: 0,
  ok: 1
}
```

Рис. 2: Вызов метода db.stats()

Метод `stats()` используется для получения статистики о базе данных или коллекции. Как видим, из-за того, что бд `test` пустая, то статистики нет.

Для создания базы данных использовалась команда:

```
use learn
```

Команда `use` переключает текущий контекст на указанную базу данных.

Для просмотра списка всех существующих БД была использована команда:

```
show dbs
```

```
}  
[test> use learn  
switched to db learn  
[learn> show dbs  
admin    40.00 KiB  
config  60.00 KiB  
local    40.00 KiB
```

Рис. 3: Переключение контекста на новую БД и выполнения команды show dbs

В результате выполнения команды выводятся имена баз данных и объем занимаемого ими дискового пространства.

В списке отображаются учебные и системные базы данных MongoDB Atlas. Базы данных с префиксом `sample_` являются демонстрационными наборами данных, автоматически добавляемыми MongoDB Atlas для обучения и тестирования запросов.

Системные базы данных:

admin — Системная база данных MongoDB, предназначенная для хранения информации о пользователях, ролях и административных настройках сервера.

local — Системная база данных, используемая MongoDB для хранения служебной информации replica set и внутренних данных сервера.

База данных `learn` ещё не отображалась в списке, поскольку MongoDB создает базы данных физически только после добавления первого документа или коллекции.

4.3 Работа с коллекциями и документами

Для создания коллекции unicorns была использована такая команда:

```
db.unicorns.insertOne({name: 'Aurora', gender: 'f',  
weight: 450})
```

```
learn> db.unicorns.insertOne({  
|   name: 'Aurora',  
|   gender: 'f',  
|   weight: 450  
| })  
|  
|  
{  
  acknowledged: true,
```

Рис. 4: Создание коллекции unicorns

Для просмотра коллекций в БД была использована команда:

```
show collections
```

```
learn> show collections  
unicorns
```

Рис. 5: Получение списка коллекций

Для переименования коллекции была использована команда:

```
db.unicorns.renameCollection("horses")
```

```
learn>  
[| db.unicorns.renameCollection("horses")  
{ ok: 1 }
```

Рис. 6: Успешное переименование коллекции

Для получения статистики коллекции была использована команда:

```
db.horses.stats()
```

```
[learn> db.horses.stats()
{
  ok: 1,
  capped: false,
  wiredTiger: {
    metadata: { formatVersion: 1 },
    creationString: 'access_pattern_hint=none,allocation_size=4KB,app_metadata=(formatVersion=1),assert=(commit_timestamp=off),block_allocation=best,block_compressor=snappy,cache_resident=false,checksum=on,colgroups=,collator=none,exclusive=false,extractor=,format=btree,huffman_key=,huffman_value=,ignore_in_memory_cache_size=false,immutable=false,insert_order=1,lsn=,file_metadata=,metadata_file=,panic_corrupt=true,repair=false),internal_item_max=0,internal_key_max=0,internal_key_min=0,internal_value_max=0,leaf_key_max=0,leaf_page_max=32KB,leaf_value_max=64MB,log=(enabled=true),lsm=(auto_throttle=1,auto_throttle_max=1000000),bloom_oldest=,chunk_count_limit=,chunk_max=,chunk_size=,merge_custom=(prefix=,start_generation=,suffix=),merge_max=1000,os_cache_dirty_max=0,os_cache_max=0,prefix_compression=false,prefix_compression_min=4,source=,split_deepen_min_child=,storage=(auth_token=,bucket=,bucket_prefix=,cache_directory=,local_retention=300,name=,object_target_size=0),type=file'
  }
}
```

Рис. 7: Успешное получение статистики коллекции

Для удаления коллекции была использована команда:

```
db.horses.drop()
```

Для удаления БД была использована команда:

```
db.dropDatabase()
```

```
[learn> db.horses.drop()
true
[learn> db.dropDatabase()
{ ok: 1, dropped: 'learn' }
learn>
```

Рис. 8: Успешное удаление коллекции и БД

5 Выполнение лабораторной работы №6.2

5.1 Вставка документов в коллекцию

Выполнение практического задания 2.1.1

С помощью команды use была создана БД learn. После этого, используя первый способ для добавления документов в коллекцию из описания ЛР 6.2, в коллекцию unicorns были добавлены документы:

```
db.unicorns.insert({name: 'Horny', loves: ['carrot', 'papaya'],
weight: 600, gender: 'm', vampires: 63});
```

```

db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'],
weight: 450, gender: 'f', vampires: 43});
db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'],
weight: 984, gender: 'm', vampires: 182});
db.unicorns.insert({name: 'Roooooodles', loves: ['apple'], weight:
575, gender: 'm', vampires: 99});
db.unicorns.insert({name: 'Solnara', loves:['apple', 'carrot',
'chocolate'], weight:550, gender:'f', vampires:80});
db.unicorns.insert({name:'Ayna', loves: ['strawberry', 'lemon'],
weight: 733, gender: 'f', vampires: 40});
db.unicorns.insert({name:'Kenny', loves: ['grape', 'lemon'], weight:
690, gender: 'm', vampires: 39});
db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'],
weight: 421, gender: 'm', vampires: 2});
db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'],
weight: 601, gender: 'f', vampires: 33});
db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'],
weight: 650, gender: 'm', vampires: 54});
db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'],
weight: 540, gender: 'f'});

```

```

learn> db.unicorns.insertMany([
| {name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63},
| {name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43},
| {name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182},
| {name: 'Roooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99},
| {name: 'Solnara', loves:['apple', 'carrot', 'chocolate'], weight:550, gender:'f', vampires:80},
| {name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40},
| {name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39},
| {name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2},
| {name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33},
| {name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54},
| {name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'}
| ])

```

Рис. 9: Добавление документа в коллекцию первым способом

И также используя второй способ из описания, был добавлен еще один документ:


```
-
learn> document = {
|   name: 'Dunx',
|   loves: ['grape', 'watermelon'],
|   weight: 704,
|   gender: 'm',
|   vampires: 165
[| }
{
  name: 'Dunx',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
learn> █
```

Рис.10: Добавление документа в коллекцию вторым способом

После этого была сделана проверка содержимого коллекции с помощью метода `find()`:

```

{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a1860641d04b9d98b695ce8'),
    '1': ObjectId('6a1860641d04b9d98b695ce9'),
    '2': ObjectId('6a1860641d04b9d98b695cea'),
    '3': ObjectId('6a1860641d04b9d98b695ceb'),
    '4': ObjectId('6a1860641d04b9d98b695cec'),
    '5': ObjectId('6a1860641d04b9d98b695ced'),
    '6': ObjectId('6a1860641d04b9d98b695cee'),
    '7': ObjectId('6a1860641d04b9d98b695cef'),
    '8': ObjectId('6a1860641d04b9d98b695cf0'),
    '9': ObjectId('6a1860641d04b9d98b695cf1'),
    '10': ObjectId('6a1860641d04b9d98b695cf2')
  }
}
learn>

```

Рис. 11: Проверка содержимого коллекции с помощью метода find

5.2 Выборка данных из БД

Выполнение практического задания 2.2.1

В ходе выполнения задания были сформированы запросы выборки документов из коллекции `unicorns` с использованием метода `find()`.

- 1) Получение списка самцов:

```
[learn> db.unicorns.find({gender: 'm'}).sort({name: 1})
[
  {
    _id: ObjectId('6a1860911d04b9d98b695cf3'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a1860641d04b9d98b695ce8'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a1860641d04b9d98b695cee'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a1860641d04b9d98b695cf1'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a1860641d04b9d98b695cef'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a1860641d04b9d98b695ceb'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm'
  }
]
```

Рис. 12: Список всех самцов с сортировкой по имени

Запрос выполняет выборку всех самцов единорогов и сортирует результат по имени в алфавитном порядке.

2) Получение первых трех самок:

```
learn>
[| db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
[
  {
    _id: ObjectId('6a1860641d04b9d98b695ce9'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a1860641d04b9d98b695ced'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a1860641d04b9d98b695cf0'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
```

Рис. 13: Список первых трех самок с сортировкой по имени

Запрос выводит только первые три документа после сортировки списка самок по имени.

3) Получение всех самок, любящих carrot:


```

]
learn> db.unicorns.find({gender: 'f', loves: 'carrot'})
[
  {
    _id: ObjectId('6a1860641d04b9d98b695ce9'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a1860641d04b9d98b695cec'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a1860641d04b9d98b695cf2'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]

```

Рис. 14: Список всех самок, любящих carrot

4) Получение только первой такой самки через findOne:

```

learn>
[| db.unicorns.findOne({gender: 'f', loves: 'carrot'})
{
  _id: ObjectId('6a1860641d04b9d98b695ce9'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
learn>

```

Рис. 15: Первая самка, любящая carrot, через findOne

5) Получение только первой такой самки через limit:

```
[learn> db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
[
  {
    _id: ObjectId('6a1860641d04b9d98b695ce9'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```

Рис. 16: Первая самка, любящая carrot, через limit

Выполнение практического задания 2.2.2

Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях.

```
]
[learn> db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0}).sort({name: 1})
[
  {
    _id: ObjectId('6a1860911d04b9d98b695cf3'),
    name: 'Dunx',
    weight: 704,
    vampires: 165
  },
  {
    _id: ObjectId('6a1860641d04b9d98b695ce8'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('6a1860641d04b9d98b695cee'),
    name: 'Kenny',
    weight: 690,
    vampires: 39
  },
  {
    _id: ObjectId('6a1860641d04b9d98b695cf1'),
    name: 'Pilot',
    weight: 650,
    vampires: 54
  },
  {
    _id: ObjectId('6a1860641d04b9d98b695cef'),
    name: 'Raleigh',
    weight: 421,
    vampires: 2
  },
  {
    _id: ObjectId('6a1860641d04b9d98b695ceb'),
    name: 'Rooooooodles',
    weight: 575,
    vampires: 99
  },
  {
    _id: ObjectId('6a1860641d04b9d98b695cea'),
    name: 'Unicrom',
    weight: 984,
    vampires: 182
  }
]
```

Рис. 17: Список самцов, исключая информацию о предпочтениях

Выполнение практического задания 2.2.3

Вывести список единорогов в обратном порядке добавления.

```
,
learn>
| db.unicorns.find().sort({$natural: -1})
|
{
  _id: ObjectId('6a1860911d04b9d98b695cf3'),
  name: 'Dunx',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
},
{
  _id: ObjectId('6a1860641d04b9d98b695cf2'),
  name: 'Nimue',
  loves: [ 'grape', 'carrot' ],
  weight: 540,
  gender: 'f'
},
{
  _id: ObjectId('6a1860641d04b9d98b695cf1'),
  name: 'Pilot',
  loves: [ 'apple', 'watermelon' ],
  weight: 650,
  gender: 'm',
  vampires: 54
},
{
  _id: ObjectId('6a1860641d04b9d98b695cf0'),
  name: 'Leia',
  loves: [ 'apple', 'watermelon' ],
  weight: 601,
  gender: 'f',
```

Рис. 18: Список единорогов в обратном порядке добавления

Выполнение практического задания 2.2.4

Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

```
]
learn> db.unicorns.find({}, {_id: 0, name: 1, loves: {$slice: 1}})
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Nimue', loves: [ 'grape' ] },
  { name: 'Dunx', loves: [ 'grape' ] }
]
learn> █
```

Рис. 19: Список единорогов с названием первого предпочтения, исключая поле `_id`

В ходе выполнения задания была выполнена выборка документов из коллекции `unicorns` с использованием проекции полей.

Для массива `loves` был применен оператор `$slice`, позволяющий вывести только первый элемент массива предпочтений. Также из результата был исключен идентификатор `_id`.

5.3 Логические операторы

Выполнение практического задания 2.3.1

Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.

```
learn>
[| db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

Рис. 20: Список единорогов весом от 500 кг до 700 кг, исключая поле `_id`

В ходе выполнения задания были использованы логические операторы сравнения `$gte` и `$lte`.

Выполнение практического задания 2.3.2

Вывести список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора.

```
learn> db.unicorns.find({
|   gender: 'm',
|   weight: {$gte: 500},
|   loves: {$all: ['grape', 'lemon']}}
[| ], {_id: 0})
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
```

Рис. 21: Список самцов весом от 500 кг и предпочитающих grape и lemon, исключая поле `_id`

В ходе выполнения задания был использован оператор `$all`, предназначенный для поиска документов, массив которых содержит все указанные элементы.

Выполнение практического задания 2.3.3

Найти всех единорогов, не имеющих ключ `vampires`.

```
[learn> db.unicorns.find({vampires: {$exists: false}})
[
  {
    _id: ObjectId('6a1860641d04b9d98b695cf2'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

Рис. 22: Список единорогов, не имеющих поле `vampires`

В ходе выполнения задания был использован оператор `$exists`, позволяющий проверить наличие или отсутствие поля в документе.

Выполнение практического задания 2.3.4

Вывести список упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.

```
learn> db.unicorns.find(
|   {gender: 'm'},
|   {_id: 0, name: 1, loves: {$slice: 1}}
| ).sort({name: 1})
[
  { name: 'Dunx', loves: [ 'grape' ] },
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Roooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]
```

Рис. 23: Список самцов с информацией об их первом предпочтении, сортировка по имени

5.4 Запросы к вложенным объектам

Выполнение практического задания 3.1.1

1) В ходе выполнения задания была создана коллекция `towns`, содержащая вложенные документы `mayor`. Были добавлены сведения о городах, населении, известных особенностях и мэрах.


```

learn>
| db.towns.insertMany([
|   {
|     name: "Punxsutawney",
|     population: 6200,
|     last_sensus: ISODate("2008-01-31"),
|     famous_for: ["phil the groundhog"],
|     mayor: {name: "Jim Wehrle"}
|   },
|   {
|     name: "New York",
|     population: 22200000,
|     last_sensus: ISODate("2009-07-31"),
|     famous_for: ["status of liberty", "food"],
|     mayor: {name: "Michael Bloomberg", party: "I"}
|   },
|   {
|     name: "Portland",
|     population: 528000,
|     last_sensus: ISODate("2009-07-20"),
|     famous_for: ["beer", "food"],
|     mayor: {name: "Sam Adams", party: "D"}
|   }
| ])
| {
|   acknowledged: true,
|   insertedIds: {
|     '0': ObjectId('6a1864191d04b9d98b695cf4'),
|     '1': ObjectId('6a1864191d04b9d98b695cf5'),
|     '2': ObjectId('6a1864191d04b9d98b695cf6')
|   }
| }

```

Рис. 24: Создание коллекции towns и добавление документов

2) Сформировать запрос, который возвращает список городов с независимыми мэрами (party="I"). Вывести только название города и информацию о мэре.

```

learn> db.towns.find(
|   {"mayor.party": "I"},
|   {_id: 0, name: 1, mayor: 1}
| )
|

```

Рис. 25: Список городов с независимыми мэрами

В ходе выполнения задания был сформирован запрос к вложенному объекту с использованием оператора точки. Были выбраны города, мэры которых являются независимыми (`party = "I"`).

3) Сформировать запрос, который возвращает список беспартийных мэров (`party` отсутствует) . Вывести только название города и информацию о мэре.

```
| db.towns.find(  
|   {"mayor.party": {$exists: false}},  
|   {_id: 0, name: 1, mayor: 1}  
| )  
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]  
learn>
```

Рис. 26: Список городов с беспартийными мэрами

В ходе выполнения задания был использован оператор `$exists` для поиска документов, в которых отсутствует поле `party` у вложенного объекта `mayor`.

Выполнение практического задания 3.1.2

- 1) *Сформировать функцию для вывода списка самцов единорогов.*
- 2) *Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке.*
- 3) *Вывести результат, используя `forEach`*

Функция для вывода самцов:

```
learn> fn = function() {  
|   return this.gender == "m";  
| }  
[Function: fn]
```

Рис. 27: Создание и вызов функции для вывода списка единорогов самцов

В ходе выполнения задания была создана JavaScript-функция для получения списка самцов единорогов.

Создание курсора:

```
[learn> var cursor = db.unicorns.find(fn).sort({name: 1}).limit(2);
```

Рис. 28: Создание курсора на основе функции

Вывод через `forEach`:

```
learn> cursor.forEach(function(obj) {  
|   print(obj.name);  
| })
```

Рис. 29: Вывод результата с помощью метода `forEach`

На основе функции был создан курсор с сортировкой по имени и ограничением количества документов. Для вывода результатов использовался метод `forEach()`.

5.5 Агрегированные запросы

Выполнение практического задания 3.2.1

Вывести количество самок единорогов весом от полутонны до 600 кг.

```
learn> db.unicorns.find({
|   gender: 'f',
|   weight: {$gte: 500, $lte: 600}
| })
[ ]
2
```

Рис. 30: Вывод количества самок единорогов весом от 500 кг и до 600 кг

Выполнение практического задания 3.2.2

Вывести список уникальных предпочтений единорогов.

```
learn> db.unicorns.distinct("loves")
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
```

Рис. 31: Вывод списка уникальных предпочтений единорогов

В ходе выполнения задания был использован метод `distinct()` для получения уникальных значений предпочтений единорогов.

Выполнение практического задания 3.2.3

Посчитать количество особей единорогов обоих полов.

```
learn> db.unicorns.aggregate([
|   {$group: {_id: "$gender", count: {$sum: 1}}}
| ])
[ { _id: 'f', count: 5 }, { _id: 'm', count: 7 } ]
learn> █
```

Рис. 32: Вывод количества самцов и самок

В ходе выполнения задания был использован агрегатный запрос `aggregate()` с оператором `$group` для подсчета количества документов по полю `gender`.

5.6 Редактирование данных

Выполнение практического задания 3.3.1

1. *Выполнить команду:*

```
> db.unicorns.save({name: 'Barney', loves: ['grape'],
weight: 340, gender: 'm'})
```

2. *Проверить содержимое коллекции `unicorns`.*

В более новой версии MongoDB не был найден метод `save()`

Выполнение практического задания 3.3.2

1. *Для самки единорога Айна внести изменения в БД: теперь ее вес 800, она убила 51 вампира.*
2. *Проверить содержимое коллекции `unicorns`.*

```
learn> db.unicorns.updateOne(
|   {name: 'Ayna'},
|   {$set: {weight: 800, vampires: 51}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Рис. 33: Изменение документа и проверка содержимого

В ходе выполнения задания был использован оператор `$set` для изменения отдельных полей документа.

Выполнение практического задания 3.3.3

1. Для самца единорога *Raleigh* внести изменения в БД: теперь он любит рэдбул.
2. Проверить содержимое коллекции *unicorns*.

```
learn> db.unicorns.find({name: 'Raleigh'})
[
  {
    _id: ObjectId('6a1860641d04b9d98b695cef'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]
```

Рис. 34: Изменение документа и проверка содержимого

В ходе выполнения задания был использован оператор `$push` для добавления нового элемента в массив `loves`.

Выполнение практического задания 3.3.4

1. Всем самцам единорогов увеличить количество убитых вампиров на 5.

2. Проверить содержимое коллекции `unicorns`.

```
[learn> db.unicorns.find({gender: 'm'})
[
  {
    _id: ObjectId('6a1860641d04b9d98b695ce8'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 68
  },
  {
    _id: ObjectId('6a1860641d04b9d98b695cea'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 187
  },
  {
    _id: ObjectId('6a1860641d04b9d98b695ceb'),
    name: 'Roooooodles',
    loves: [ 'apple' ],
    weight: 575
  }
]
```

Рис. 35: Изменение документа и проверка содержимого

В ходе выполнения задания был использован оператор `$inc` для увеличения числового значения поля `vampires`.

Выполнение практического задания 3.3.5

1. Изменить информацию о городе Портланд: мэр этого города теперь беспартийный.
2. Проверить содержимое коллекции `towns`.


```

learn> db.towns.updateOne(
|   {name: 'Portland'},
|   {$unset: {"mayor.party": 1}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}

```

Рис. 36: Изменение документа и проверка содержимого

В ходе выполнения задания был использован оператор `$unset` для удаления поля `party` из вложенного объекта `mayor`.

Выполнение практического задания 3.3.6

1. Изменить информацию о самце единорога *Pilot*: теперь он любит и шоколад.
2. Проверить содержимое коллекции *unicorns*.

```

learn> db.unicorns.updateOne(
|   {name: 'Pilot'},
|   {$push: {loves: 'chocolate'}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}

```

```
[learn> db.unicorns.find({name: 'Pilot'})
[
  {
    _id: ObjectId('6a1860641d04b9d98b695cf1'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
```

Рис. 37: Изменение документа и проверка содержимого

В ходе выполнения задания был использован оператор `$push` для добавления нового элемента в массив предпочтений.

Выполнение практического задания 3.3.7

1. Изменить информацию о самке единорога Aurora: теперь она любит еще и сахар, и лимоны.
2. Проверить содержимое коллекции `unicorns`.

```
learn> db.unicorns.updateOne(
|   {name: 'Aurora'},
|   {$addToSet: {loves: {$each: ['sugar', 'lemon']}}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}

[learn> db.unicorns.find({name: 'Aurora'})
[
  {
    _id: ObjectId('6a1860641d04b9d98b695ce9'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```

Рис. 38: Изменение документа и проверка содержимого

В ходе выполнения задания был использован оператор `$addToSet` совместно с `$each` для добавления нескольких уникальных элементов в массив.

5.7 Удаление данных

Выполнение практического задания 3.4.1

1) Удаление городов с беспартийными мэрами

```
]
[learn> db.towns.remove({"mayor.party": {"$exists": false}})
DeprecationWarning: Collection.remove() is deprecated. Use deleteOne, deleteMany, findOneAndDelete, or bulkWrite.
{ acknowledged: true, deletedCount: 2 }
[learn> ]
```

Рис. 39: Удаление данных и проверка содержимого

2) Очистка коллекции towns

```
[learn> db.towns.find()
[
  {
    _id: ObjectId('6a1864191d04b9d98b695cf5'),
    name: 'New York',
    population: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  }
]
```

Рис. 40: Очистка коллекции и проверка содержимого

3) Просмотр коллекций

```
{ acknowledged: true, deletedCount:
[learn> show collections
towns
unicorns
-
```

Рис. 41: Получение списка коллекций

В ходе выполнения задания была выполнена полная очистка коллекции `towns` и просмотр списка существующих коллекций базы данных.

5.8 Ссылки в MongoDB

Выполнение практического задания 4.1.1

- 1) Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.

- 2) Включите для нескольких единорогов в документы ссылку на зону обитания, используя второй способ автоматического связывания.
- 3) Проверьте содержание коллекции единорогов.

1) Создание коллекции habitats

```
-----
learn> db.habitats.drop()
true
learn>
| db.habitats.insertMany([
|   {
|     _id: "forest",
|     full_name: "Magic Forest",
|     description: "Forest habitat for unicorns"
|   },
|   {
|     _id: "mountains",
|     full_name: "Crystal Mountains",
|     description: "Mountain habitat for unicorns"
|   },
|   {
|     _id: "lake",
|     full_name: "Silver Lake",
|     description: "Lake habitat for unicorns"
|   }
| ])
{
  acknowledged: true,
  insertedIds: { '0': 'forest', '1': 'mountains', '2': 'lake' }
}
----- ■
```

Рис. 42: Создание коллекции habitats

В ходе выполнения задания была создана коллекция **habitats**, содержащая информацию о зонах обитания единорогов.

2) Добавление ссылок

```
-
learn> db.unicorns.updateOne(
|   {name: "Horny"},
|   {$set: {habitat: {$ref: "habitats", $id: "mountains"}}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
---
```

```
learn> db.unicorns.updateOne(
|   {name: "Nimue"},
|   {$set: {habitat: {$ref: "habitats", $id: "lake"}}}
| )
[|
| {
|   acknowledged: true,
|   insertedId: null,
|   matchedCount: 1,
|   modifiedCount: 1,
|   upsertedCount: 0
| }
| ]
```

Рис. 43: Добавление ссылок для двух единорогов

3) Проверка содержимого коллекции

```
learn> db.unicorns.find(
|   {habitat: {$exists: true}},
|   {_id: 0, name: 1, habitat: 1}
| )
[|
| { name: 'Horny', habitat: DBRef('habitats', 'mountains') },
| { name: 'Aurora', habitat: DBRef('habitats', 'forest') },
| { name: 'Nimue', habitat: DBRef('habitats', 'lake') }
| ]
```

Рис. 44: Проверка содержимого коллекции unicorns

В ходе выполнения задания были созданы ссылки между коллекциями с использованием механизма DBRef.

5.9 Настройка индексов

Выполнение практического задания 4.2.1

1. Проверьте, можно ли задать для коллекции *unicorns* индекс для ключа *name* с флагом *unique*.

```
learn> db.unicorns.createIndex({name: 1}, {unique: true})
name_1
-----
```

Рис. 45: Успешное создание индекса

Сейчас все имена уникальны:

- Horny
- Aurora
- Unicrom
- Barny

- Pilot
- и т.д.

Поэтому индекс должен успешно создан.

5.9 Управление индексами

Выполнение практического задания 4.3.1

- 1) *Получите информацию о всех индексах коллекции `unicorns`.*
- 2) *Удалите все индексы, кроме индекса для идентификатора.*
- 3) *Попытайтесь удалить индекс для идентификатора.*

1) Получение информации обо всех индексах

```
learn>
[| db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
```

Рис. 46: Получение списка индексов

2) Удаление всех индексов, кроме `_id`

```
learn> db.unicorns.dropIndexes()  
{  
  nIndexesWas: 2,  
  msg: 'non-_id indexes dropped for collection',  
  ok: 1  
}
```

Рис. 47: Удаление индексов и получение списка индексов

В ходе выполнения задания были удалены все пользовательские индексы коллекции `unicorns`. После удаления в коллекции остался только системный индекс `_id`.

3) Попытка удалить индекс `_id`

```
learn> db.unicorns.dropIndex("_id")  
MongoServerError[InvalidOptions]: cannot drop _id index  
learn>
```

Рис. 48: Получение ошибки при попытке удалить системный индекс `_id`

5.9 План запроса

Выполнение практического задания 4.4.1

- 1) Создайте объемную коллекцию `numbers`, задействовав курсор:

```
for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}
```
- 2) Выберите последних четыре документа.
- 3) Проанализируйте план выполнения запроса 2. Сколько потребовалось времени на выполнение запроса? (по значению параметра `executionTimeMillis`)
- 4) Создайте индекс для ключа `value`.
- 5) Получите информацию о всех индексах коллекции `numbers`.
- 6) Выполните запрос 2.
- 7) Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса?
- 8) Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?

- 1) Создание объемной коллекции `numbers`


```

learn> for (i = 0; i < 100000; i++) {
|   db.numbers.insertOne({value: i})
| }

{
  acknowledged: true,
  insertedId: ObjectId('6a1869321d04b9d98b6ae397')
}

```

Рис. 49: Создание коллекции numbers

2) Выборка последних четырех документов

```

learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a1869321d04b9d98b6ae397'), value: 99999 },
  { _id: ObjectId('6a1869321d04b9d98b6ae396'), value: 99998 },
  { _id: ObjectId('6a1869321d04b9d98b6ae395'), value: 99997 },
  { _id: ObjectId('6a1869321d04b9d98b6ae394'), value: 99996 }
]

```

Рис. 50: Получение последних 4 документов

В ходе выполнения задания был выполнен запрос для получения последних четырех документов коллекции numbers.

3) Анализ плана выполнения запроса без индекса

```

learn> db.numbers.explain("executionStats").find().sort({value: -1}).limit(4)
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'AE5753F2',
    planCacheShapeHash: 'AE5753F2',
    planCacheKey: 'AE69D0D4',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'SORT',
      sortPattern: { value: -1 },
      memLimit: 104857600,

```

Рис. 51: Получение анализа запроса без индекса

В поле executionTimeMillis значение 128, значит время выполнения запроса составило: 128ms.

4) Создание индекса по полю value

```
[learn> db.numbers.createIndex({value: 1})
value_1
100000 ■
```

Рис. 52: Создание индекса по полю value

5) Получение информации об индексах

```
[learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
```

Рис. 53: Получение списка индексов

6) Повторное выполнение запроса

```
]
[learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a1869321d04b9d98b6ae397'), value: 99999 },
  { _id: ObjectId('6a1869321d04b9d98b6ae396'), value: 99998 },
  { _id: ObjectId('6a1869321d04b9d98b6ae395'), value: 99997 },
  { _id: ObjectId('6a1869321d04b9d98b6ae394'), value: 99996 }
]
```

Рис. 54: Повторное выполнение запроса

7) Анализ плана запроса с индексом

```
[learn> db.numbers.explain("executionStats").find().sort({value: -1}).limit(4)
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'AE5753F2',
    planCacheShapeHash: 'AE5753F2',
    planCacheKey: 'AE69D0D4',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'LIMIT',
      limitAmount: 4,
      inputStage: {
        stage: 'FETCH',
        inputStage: {
          stage: 'IXSCAN',
          keyPattern: { value: 1 },
          indexName: 'value_1',
          isMultiKey: false,
          multiKeyPaths: { value: [] },

```

Рис. 55: Получение анализа запроса с индексом

Теперь в поле `executionTimeMillis` значение 0, значит время выполнения запроса составило: 0ms за счет работы индекса.

Более эффективным является запрос с использованием индекса, так как MongoDB выполняет поиск по индексированной структуре данных без полного просмотра всей коллекции.

Вывод

В ходе лабораторной работы были выполнены основные операции MongoDB: создание базы данных и коллекций, вставка документов, выборка с условиями, сортировка, ограничение результата, работа с массивами, вложенными объектами и проекциями.

Были применены логические операторы, агрегированные запросы, операции обновления и удаления документов. Также были созданы ссылки между коллекциями, настроены индексы и выполнен анализ плана запроса. В процессе выполнения учтены отличия текущей версии `mongosh` от устаревшего синтаксиса методических материалов: вместо `save` использован `insertOne` или `replaceOne` с `upsert`, вместо `remove` - `deleteMany`, вместо `ensureIndex` - `createIndex`.